

DevLens (now **Acuity**) Proposal

Sithu Soe - A17342422

Phyo Thant - A18498144

Planned technologies

Proposed	Status	Where / Notes
Next.js (frontend)	✅ Implemented as written	App Router project in frontend/ . Entry at frontend/app/layout.tsx .
TailwindCSS	✅ Implemented as written	Used for layout/spacing; design tokens (OKLCH custom properties) live in frontend/app/globals.css .
Monaco Editor (sandbox code interface)	✅ Implemented as written	@monaco-editor/react in frontend/components/Editor/CodeEditor.tsx , plus the multi-file variant in frontend/components/Editor/MultiFileEditor.tsx .
FastAPI (Python backend)	✅ Implemented as written	App factory in backend/app/main.py ; routers in backend/app/routers/ .
Native Python asyncio WebSockets	✅ Implemented as written	The WS gateway is backend/app/routers/ws.py — pure FastAPI/asyncio, no Socket.IO layer.
LangGraph / LangChain	✅ Implemented as written	Agentic chain lives in backend/app/services/agent.py using LangGraph + langchain-anthropic . The proposal hedged ("if we deem necessary") — we kept it.
PostgreSQL	✅ Implemented as written	SQLAlchemy 2.0 async + asyncpg; models in backend/app/db/models.py ; migrations in backend/app/migrations/ (0001 → 0007). Provided by Supabase.
Redis (WebSocket state, pub/sub, rate limiting)	✅ Implemented as written for state + pub/sub. Rate limiting partially landed as token budgeting (Redis INCRBY on the AI token counter in backend/app/routers/ws.py). Per-IP rate limiting is not implemented.	
Supabase Auth	✅ Implemented as written	Verified server-side in backend/app/security.py (ES256/RS256 via JWKS, HS256 fallback for dev).
Render (deployment)	🟡 Planned	Codebase is deployment-ready (docker-compose.yml for local, env-driven config). No

Proposed	Status	Where / Notes
		<code>render.yaml</code> has been committed yet — see SETUP.md and plan.md §6.
LLM provider	Extra detail not in proposal	Anthropic Claude via <code>langchain-anthropic</code> (backend/app/services/llm.py); model id env-driven (<code>ANTHROPIC_MODEL</code>).
Package managers	Extra detail not in proposal	<code>uv</code> for the Python backend, <code>pnpm</code> for the Next.js frontend.

First deliverable

"A functional live interview... candidate and interviewer in a shared active room... candidate uses the AI Assistant... interviewer + AI evaluate the candidate... when the interview ends, the admin dashboard is populated with a summary/metrics."

Goal	Status	Where / Notes
Shared active interview session (candidate + interviewer)	✓ Implemented as written	Sessions created via frontend/components/CreateSessionForm.tsx ; join flow in frontend/app/join/[code]/ ; live candidate IDE at frontend/app/interview/[sessionId]/page.tsx ; interviewer mirror at frontend/app/dashboard/[sessionId]/page.tsx .
Candidate uses the AI Assistant inside the IDE	✓ Implemented as written	Chat UI in frontend/components/Chat/ChatBox.tsx ; agent in backend/app/services/agent.py .
Live evaluation of the candidate by the interviewer + AI	✓ Implemented as written	Hidden interviewer dashboard mirrors the candidate's code/chat (frontend/app/dashboard/[sessionId]/page.tsx); AI evaluation lives in the scorecard (backend/app/services/scorecard.py) plus opt-in push-back hints (backend/app/services/pushback.py).
Summary/metrics populated on interview end	✓ Implemented as written	<code>interview_end</code> WS event fires the scorecard generator; results render in frontend/components/Dashboard/SummaryView.tsx (radar chart, dimension bars, AI summary, replay timeline).

Beyond the literal proposal text, the first deliverable also shipped:

- Auth + role-routed dashboards ([frontend/app/\(auth\)/](#)).
- A 4-step session-creation wizard with drag-drop file/folder upload.
- A pre-interview waiting room with admit/kick gates (`session_participants.admitted`).
- Per-session display-name gating ([frontend/components/DisplayNameModal.tsx](#)).

What remains for first deliverable to be **shippable**: a live run against real Supabase + Anthropic credentials and a Render deploy — see [plan.md](#) §6 / §8.

Rough architecture for first deliverable

1. Auth & Session Manager (Supabase & Postgres)

- **Inputs (login credentials, role assignment):** ✅ Implemented — [frontend/app/\(auth\)/login/page.tsx](#), [frontend/app/\(auth\)/signup/page.tsx](#); [profiles](#) table in [backend/app/db/models.py](#) carries the role enum.
- **Outputs (JWT session tokens, InterviewRoom config):** ✅ Implemented — Supabase issues JWTs; backend verifies them in [backend/app/security.py](#). The "InterviewRoom" is now the [interview_sessions](#) row (renamed Room → Session in migration [0003](#)) carrying language, starting code, guardrails, hallucination %, token budget — see [backend/app/db/models.py](#).
- **Effects (secures routes, initializes DB entries):** ✅ Implemented — [Depends\(get_current_user\)](#) guards on the routers; session row created in [backend/app/routers/interview.py](#).

2. Browser IDE Component (Next.js & Monaco Editor)

- **Inputs (candidate inputs):** ✅ Implemented — [frontend/components/Editor/CodeEditor.tsx](#) (single-file) + [frontend/components/Editor/MultiFileEditor.tsx](#) (multi-file projects, added post-D1).
- **Outputs (CodeChange + ChatMessage WS payloads):** ✅ Implemented — debounced in the candidate page; the WS client is [frontend/lib/ws.ts](#).
- **Effects (render IDE + push telemetry):** ✅ Implemented — extended beyond the proposal with [tab_switch](#), [cursor_move](#), [mouse_move](#), and [shell_command](#) events (see [backend/app/routers/ws.py](#)).

3. Real-Time Sync Gateway (FastAPI & Redis)

- **Inputs (WS payloads from candidate IDE):** ✅ Implemented — [backend/app/routers/ws.py](#).
- **Outputs (Redis pub/sub broadcasts on [session:{id}](#) channels):** ✅ Implemented as written — channel naming convention documented in [CLAUDE.md](#) and [plan.md](#) §5.
- **Effects (sync state to interviewer with minimal latency):** ✅ Implemented — the interviewer's dashboard subscribes and applies events identically. Presence is Redis-tracked ([connected:{session_id}](#) set), not DB-tracked.

4. Agentic Chat Engine (LangGraph/LangChain)

- **Inputs (chat query + code state + guardrails):** ✅ Implemented — [backend/app/services/agent.py](#). Guardrails are multi-select; [build_guardrail_system](#) stacks any number of presets.
- **Outputs (LLM response payload, text + code):** ✅ Implemented — streamed back as [ai_response](#) WS events; markdown rendered in [frontend/components/Chat/ChatBox.tsx](#).
- **Effects (Copilot-style assistance honoring guardrails):** ✅ Implemented — includes a "no solutions / hints only" preset as proposed.

5. Hallucination Injector Module

- **Inputs (correct agent answer + hallucination probability):** ✅ Implemented — [backend/app/services/hallucinator.py](#); [hallucination_pct](#) column on [interview_sessions](#).
- **Outputs (subtly altered answer with logical/syntax flaws):** ✅ Implemented as written — second Claude pass that rewrites the original answer to inject a plausible flaw.
- **Effects (forces candidate to debug rather than copy):** ✅ Implemented — interviewer side sees [was_hallucinated](#) flag; candidate side has it stripped in [_pump_redis_to_ws](#).

6. Telemetry Logger

- **Inputs (timestamps, diffs, chat transcripts, eval flags):** ✅ Implemented — [events](#), [transcripts](#), [scorecards](#) tables.

- **Outputs (structured rows in EventLogs/Transcripts):**  Implemented — see [backend/app/db/models.py](#); writes go through [backend/app/services/telemetry.py](#).
- **Effects (async, non-blocking on the WS path):**  Implemented as written — `telemetry.fire(coro)` pattern keeps long-running writes off the inbound socket loop.

7. Scorecard Generator

- **Inputs (full transcript + event logs at session end):**  Implemented — [backend/app/services/scorecard.py](#).
- **Outputs (structured JSON grade across metrics):**  Implemented — Claude with `with_structured_output(_ScorecardResult)`. Recent improvement: per-dimension rubric + carve-outs so candidates who barely used the AI aren't punished, and fully-passing tests force `code_correctness = 10`.
- **Effects (final post-interview dashboard):**  Implemented — [frontend/components/Dashboard/SummaryView.tsx](#) renders radar + dimensions + AI summary + replay timeline + final code + key turns (markdown-rendered).

After first deliverable goals

Goal	Status	Where / Notes
Sandboxed code execution (Piston API or Dockerized)	 Implemented, but via Wandbox instead of Piston — Piston went whitelist-only on 2026-02-15. Code in backend/app/services/executor.py ; HTTP endpoint <code>POST /sessions/{id}/run</code> in backend/app/routers/interview.py . Supports both stdin mode and LeetCode-style "call mode" with a per-language harness.	
Replay component on the interviewer dashboard	 Implemented — frontend/components/Dashboard/ReplayTimeline.tsx . Renders a horizontal timeline of events (code runs, pastes, tab switches, idle gaps); idle detection looks at <code>code_change/cursor_move/mouse_move/chat_message/code_run/paste_flag</code> with a 15s threshold.	
Background push-back questions from an LLM	 Implemented — backend/app/services/pushback.py ; rendered in the interviewer's right rail in frontend/app/dashboard/[sessionId]/page.tsx . Stripped from candidate sockets so it stays interviewer-only.	
Interviewer pre-session config (hallucination %, token limits, guardrails)	 Implemented — 4-step wizard in frontend/components/CreateSessionForm.tsx ; persisted to <code>hallucination_pct</code> , <code>token_budget</code> , and <code>guardrail_presets</code> (multi-select) on <code>interview_sessions</code> .	
Chat-box query quota (limit)	 Reshaped — the proposal said "query quota" (count of prompts). That's been replaced by a token budget (<code>token_budget</code> column + Redis <code>INCRBY</code> on each LLM call) which is finer-grained — see migration	

Goal	Status	Where / Notes
candidate prompts)	0004_interview_type_token_budget which drops query_quota/ai_max_tokens. When the budget is exhausted, the candidate's composer is replaced with a disabled "AI assistance has run out" banner in frontend/components/Chat/ChatBox.tsx.	
Cheat mitigation: flag large external pastes	✅ Implemented — onPaste handler in frontend/app/interview/[sessionId]/page.tsx emits a paste_flag event; surfaced in the interviewer header pill, the telemetry sidebar, and the replay timeline.	

Things shipped beyond the proposal

These weren't in the original document but landed during development:

- **Multi-file projects** — session_files table, drag-drop upload, file tree, structural CRUD + files_dirty broadcast (backend/app/db/models.py, frontend/components/Editor/MultiFileEditor.tsx).
- **Interactive shell tab** — built-ins (ls, cat, pwd) + run/python/node/go run in backend/app/services/shell.py.
- **Tab-switch + cursor-move monitoring** — visibility listener + Monaco cursor stream; interviewer sees a blinking amber caret at the candidate's position.
- **Waiting room w/ admit gate** — server-authoritative session_participants.admitted.
- **Two-phase end-interview flow** — interview_ended fires immediately, scorecard_ready follows when Claude finishes, so the interviewer flips to summary without waiting on the LLM.
- **Acuity design-system overhaul** — OKLCH tokens, three-font stack (Instrument Serif / Geist / JetBrains Mono), reusable primitives in frontend/components/ui/ (see ROADMAP.md).

Explicitly dropped / replaced

- ❌ **Piston API** — replaced by Wandbox (Piston whitelist change). Local Piston/Judge0 noted as a swap option in backend/app/services/executor.py.
- ❌ **query_quota (per-prompt count)** — dropped in migration 0004 in favor of token_budget.
- ❌ **Project Title** renamed from "DevLens" to "Acuity"